

EC-341 Digital Systems Design

Experiment # 12

Implementation of Moore State Machine

1. Objectives:

To develop the source code for Moore machine by using VERILOG and obtain the simulation and synthesis.

2. Theory

A state machine is an abstract way of describing the behavior of a controller. The state machine description reduces the description of any controller to just four things: inputs, outputs, states, and state transition rules. The state machine description contains no details about the implementation of the controller (no flowcharts, programs, schematics, mechanical diagrams, etc.). Instead, a state machine only describes the different states that the controller can be in, the outputs in each state, and what input conditions will cause transitions between states. A state machine description can be used to describe and analyze existing systems and to aid in the design of new systems. The simplified description represented by a state machine allows us to verify proper operation and document the logical operation of a controller before proceeding with detailed design (e.g. writing a program or designing hardware).

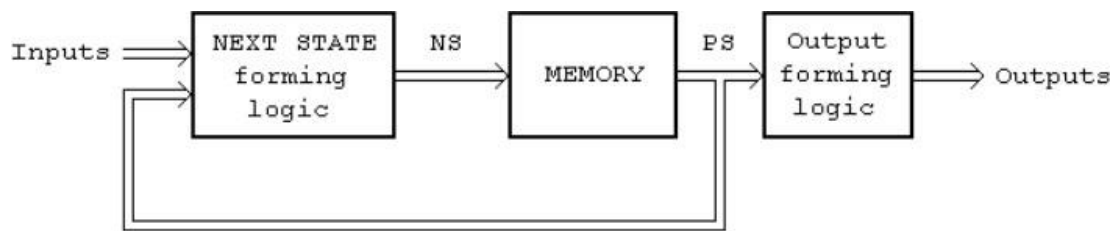
3. Design of State Machine

There are four steps to designing a state machine:

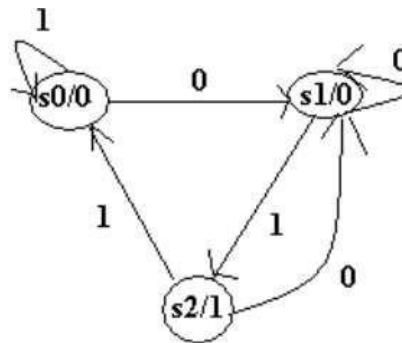
- Define the outputs – these are the output signals used by the controller
- Define the inputs – the input signals provided to the controller
- Define all of the possible states that the controller will need to be in and what values the outputs will take for each of these states
- Define the exact input conditions that will cause the state machine to switch from one state to another

3.1 Overview of Moore State Machine

A Moore machine is finite state machine (FSM) whose output depends only on the current state without depending on the current input. A Moore machine design fits the model shown below: -



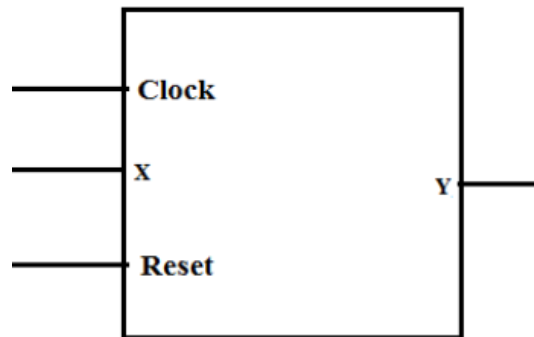
4. Example 1



A Moore machine is a finite state machine where the outputs are determined by the current state alone (and do not depend directly on the input). The state diagram for a Moore machine will include an output signal for each state. Compared with a Mealy machine, which maps transitions in the machine to outputs. The advantage of the Moore model is a simplification of the behavior.

Present State	Next State		Output	
	X = 0	X = 1	X = 0	X = 1
S0	S1	S0	0	0
S1	S1	S2	0	1
S2	S1	S0	0	0

Block Diagram is as follow: -



Verilog Code: -

```
module moore(  
    input x,  
    input clock,  
    input reset,  
    output reg y  
);  
    reg [1:0] Next_state;  
    parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10;  
    always @ (posedge clock)  
    begin  
        if (reset)  
        begin  
            Next_state = s0;  
            y = 1'b0;  
        end  
        case (Next_state)  
            s0:begin  
                y = 1'b0;  
                if (x)  
                    Next_state = s0;  
                else  
                    Next_state = s1;  
            end  
            s1:begin  
                y = 1'b0;  
                if (x)  
                    Next_state = s2;  
                else  
                    Next_state = s0;  
            end  
            s2:begin  
                y = 1'b1;  
                if (x)  
                    Next_state = s2;  
                else  
                    Next_state = s1;  
            end  
        endcase  
    end  
end
```

```

        Next_state = s1;
    end
    s2:begin
        y = 1'b1;
        if (x)
            Next_state = s0;
        else
            Next_state = s1;
        end
    endcase
end
endmodule

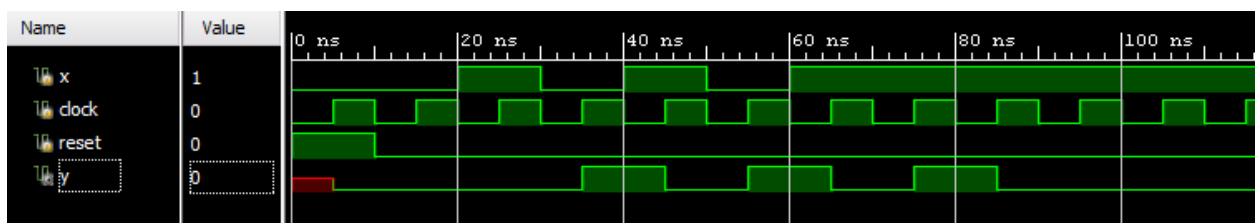
```

Test Bench: -

```

module testbench();
    reg x;
    reg clock;
    reg reset;
    wire y;
    moore uut (
        .Y(y),
        .x(x),
        .clock(clock),
        .reset(reset)
    );
    initial begin
        x = 0; clock = 0; reset = 1;
        #10 x = 0; reset = 0;
        #10 x = 1;
        #10 x = 0;
        #10 x = 1;
        #10 x = 0;
        #10 x = 1;
        #100;
    end
    always #5 clock = ~clock;
endmodule

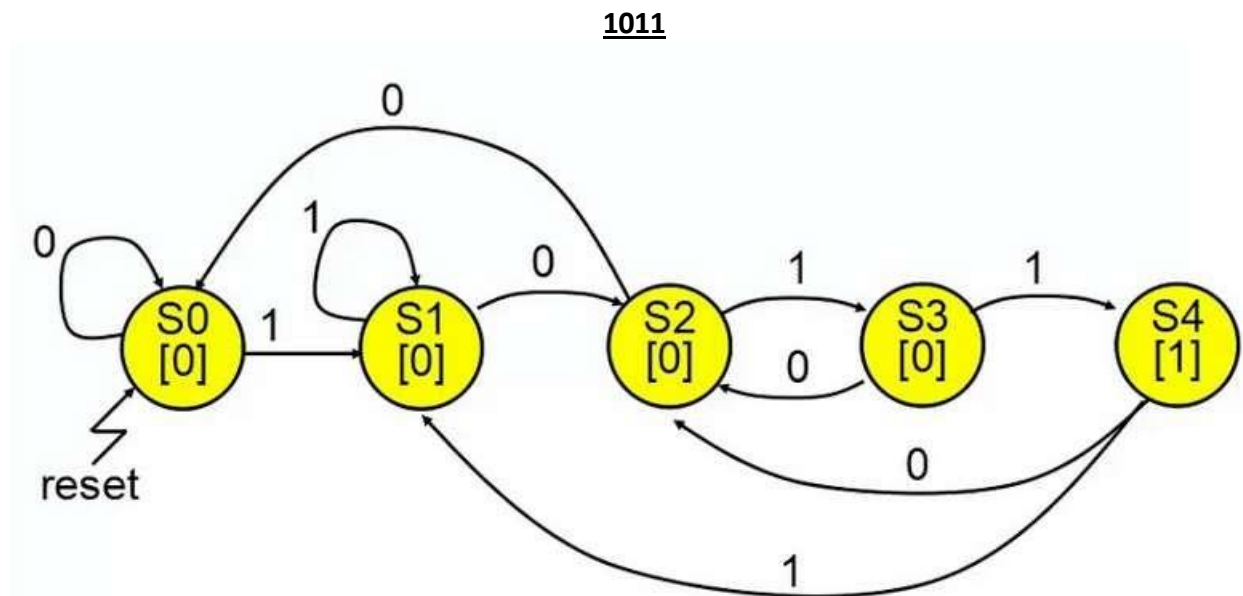
```

Output: -

5. Example 2

- a. **You are required** to design a controller for a system that actuates on the input provided by a 1-bit sensor. The system actuates (gives O/P as 1) when it detects two consecutive ones followed by a zero which are then followed by two consecutive ones.

State Diagram: -



State Table: -

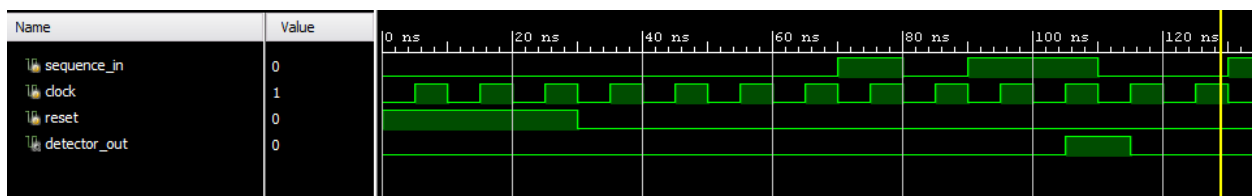
Present State (PS)	Next State (NS)		Output z	
	x=0	x=1	x=0	x=1
S0	S0	S1	0	0
S1	S2	S1	0	0
S2	S0	S3	0	0
S3	S2	S4	0	1
S4	S2	S1	0	0

Verilog Code: -

```
module moore(clock, reset, sequence_in, detector_out);
    input clock;
    input reset;
    input sequence_in;
    output reg detector_out;
    parameter Zero=3'b000,
               One=3'b001,
               OneZero=3'b011,
               OneZeroOne=3'b010,
               OneZeroOneOne=3'b110;
    reg [2:0] current_state, next_state;
    always @(posedge clock, posedge reset)
    begin
        if(reset==1)
            current_state <= Zero;
        else
            current_state <= next_state;
        end
    always @(current_state,sequence_in)
    begin
        case(current_state)
        Zero:begin
            if(sequence_in==1)
                next_state = One;
            else
                next_state = Zero;
            end
        One:begin
            if(sequence_in==0)
                next_state = OneZero;
            else
                next_state = One;
            end
        OneZero:begin
            if(sequence_in==0)
                next_state = Zero;
            else
                next_state = OneZeroOne;
            end
        OneZeroOne:begin
            if(sequence_in==0)
                next_state = OneZero;
            else
                next_state = OneZeroOneOne;
            end
        OneZeroOneOne:begin
            if(sequence_in==0)
                next_state = OneZero;
            else
                next_state = OneZeroOneOne;
            end
        end
    end
end
```

```
        else
            next_state = One;
        end
        default:next_state = Zero;
    endcase
end
always @(current_state)
begin
    case(current_state)
        Zero:    detector_out = 0;
        One:     detector_out = 0;
        OneZero: detector_out = 0;
        OneZeroOne: detector_out = 0;
        OneZeroOneOne: detector_out = 1;
        default: detector_out = 0;
    endcase
end
endmodule
```

Output: -



6. Task

1. Execute the code given in the example for overlapping/non-overlapping case.
2. You are required to write code for a basic sequential state machine using Moore based design, which detects **11011** for the following cases:
 - Overlapping
 - Non-Overlapping

(Draw complete state diagram and elaborate the table for previous, current and next state respectively)